# Diseño de casos y JUnit 5

Universidad Espíritu Santo · UCOM0310 · Semana 7
Actividad formativa 1 · 30 de septiembre de 2026

**Estudiante:** Jordy Alessandro Fajardo Figueroa
**Carrera:** Ingeniería en Ciencias de la Computación
**Materia:** Diseño de Software

## Objetivo y alcance

Diseñar casos relevantes y automatizarlos con JUnit 5 y estructura AAA. Se trabaja en un repositorio independiente de Ae6, utilizando el proyecto base original. Se prueban puedeCancelar y calcularTotal; confirmar queda para el laboratorio 2. La entrega incluye nueve casos de negocio, la prueba inicial conservada, registros reales de ejecución y un experimento de frontera.

## Matriz de casos

| ID | Entrada | Esperado | Justificación |
|---|---|---|---|
| CP-01 | Cancelar 5 h | true | Regla habitual |
| CP-02 | Cancelar 2 h | true | Detectar > en vez de >= |
| CP-03 | Cancelar 1 h | false | Frontera inferior |
| CP-04 | Cancelar 0 h | false | Sin anticipación |
| CP-05 | NORMAL, 100 | 100 | Sin descuento |
| CP-06 | VIP, 100 | 85 | Descuento del 15 % |
| CP-07 | ESTUDIANTE, 100 | 90 | Descuento del 10 % |
| CP-08 | VIP, 0 | 0 | Mínimo no negativo |
| CP-09 | NORMAL, -1 | IllegalArgumentException | Rechazar importe negativo |

La matriz completa relaciona cada caso con su nombre de prueba en docs/01_MATRIZ_CASOS_PLANTILLA.md. Fue registrada en Git antes de implementar la suite. Se conserva entornoJUnitFunciona como comprobación del arranque de JUnit; no representa una regla de negocio.

## Implementación y AAA

Cada prueba prepara entradas en Arrange, ejecuta el método en Act y comprueba el resultado en Assert. Los importes usan una tolerancia de 0.001; el total negativo exige el tipo de excepción y el mensaje Total base inválido. Los colaboradores null solo se usan porque estos métodos puros no los consultan. No se añadieron mocks ni se alteraron los esperados.

## Ejecución y experimento de frontera

Entorno: Java 21.0.12.1 y Maven 3.9.16. La línea base pasó una prueba. Tras implementar los nueve casos, mvn clean test pasó diez pruebas sin fallos, errores u omitidas. Los logs están en docs/evidencias.

El microexperimento sustituyó temporalmente horasAnticipacion >= 2 por horasAnticipacion > 2. mvn test ejecutó diez pruebas y produjo exactamente un fallo: dosHorasEsElLimitePermitido, con esperado true y obtenido false. Se restauró el archivo original y la ejecución final mvn clean test volvió a producir BUILD SUCCESS: 10 pruebas, 0 fallos, 0 errores y 0 omitidas.

## Reflexión sobre el caso límite

La prueba dosHorasEsElLimitePermitido detectó mejor el error porque usa exactamente 2, la entrada donde cambia el resultado entre >= 2 y > 2. El caso habitual de 5 horas pasa con ambas implementaciones y no detecta el defecto. Las entradas 1 y 0 también mantienen el resultado false en ambas. Esto demuestra que elegir la frontera es más eficaz que repetir únicamente entradas normales.

La regla y el esperado se mantuvieron: no se modificó ninguna aserción para ocultar el fallo. El defecto temporal no se publica. La restauración se realizó conservando los bytes originales del archivo productivo y se comprobó el diff de producción.

## Autorrevisión y límites

Se diseñaron nueve casos con justificación, se implementaron nueve pruebas de negocio con nombres descriptivos y AAA, se conservó la prueba inicial y se incluyeron casos normales, alternativos, límites e inválido. La verificación final es limpia y el repositorio excluye target y tmp.

No se validan confirmar ni sus dependencias, integración, concurrencia o políticas monetarias para NaN, infinito y redondeo. La configuración heredada genera JaCoCo, pero su interpretación no es requisito de esta actividad formativa. El foco es la coherencia entre regla, caso y aserción.

## Repositorio y evidencia

github.com/jordyfajardo-17/UEES-Formativa1-Semana7
docs/evidencias/01_linea_base.txt
docs/evidencias/02_suite_antes_experimento.txt
docs/evidencias/03_experimento_frontera.txt
docs/evidencias/04_ejecucion_final.txt
docs/02_REFLEXION_FORMATIVA1.md

## Declaración de uso de IA

OpenAI Codex asistió en lectura de requisitos, diseño de casos, implementación, ejecución del experimento, documentación y publicación. Los resultados se respaldan con logs reales. El estudiante debe revisar y comprender cada caso antes de presentar la entrega; la revisión asistida no es una evaluación independiente del docente.